

Análise versão 01

Análise do projeto ph.app.br

1. Etapas:

1. Estudar como o site OpenAI pode ajudar a criar o *Assistant API da OpenAI*.

2. Criar cadastros necessários para o projeto:

1. operadores //Pode ser a secretária virtual

1. id // tipo inteiro serial
2. nome // Char com 50 posições não pode ser nulo
3. telefone : // Char com 20 posições

2. hospitais

1. id // tipo inteiro serial
2. nome // Char com 50 posições pode ser nulo
3. telefone // Char com 20 posições pode ser nulo

3. status_da_agenda_ou_consulta

1. id // tipo inteiro serial
2. nome // Char com 20 posições pode ser nulo

4. médicos

1. id // tipo inteiro serial
2. id_operadores //Tipo Inteiro
3. nome // Char com 50 posições pode ser nulo
4. telefone // Char com 25 posições pode ser nulo
5. telefone_da_secretaria // Char com 25 posições pode ser nulo
6. login // Char com 50 posições pode ser nulo
7. senha // Char com 20 posições pode ser nulo

5. serviço_de_agendas

1. id // tipo inteiro serial
2. id_operador // tipo inteiro
3. nome // Char com 100 posições pode ser nulo
4. login // Char com 50 posições pode ser nulo
5. senha // Char com 50 posições pode ser nulo

6. convênios

1. id // tipo inteiro serial
2. id_médico // tipo inteiro
3. nome // Char com 50 posições pode ser nulo
4. login // Char com 50 posições pode ser nulo
5. senha // Char com 50 posições pode ser nulo

7. clientes

1. id // tipo inteiro serial
 2. nome // Char com 50 posições. Obs: Não pode ser nulo
 3. telefone_whatsApp; // Char com 25 posições. Obs: pode ser nulo
 4. e_mail // Char com 50 posições e pode ser nulo
 5. login // Char com 50 posições e não pode ser nulo
 6. senha // Char com 20 posições e não pode ser nulo
 7. id_convênio // tipo inteiro pode ser nulo
 8. dateTime_disponível // Tipo da data e pode ser nulo. Obs: Usado para pesquisar na agenda uma data mais próxima
 9. matricula_no_convênio // Char com 50 posições e pode ser nulo.
8. integração // Usado para ser logar
1. id // tipo inteiro serial
 2. endereço_do_site // Char com 255 posições e não pode ser nulo
 3. login // Char com 50 posições e não pode ser nulo. Obs: Login para acessar o endereço_do_site
 4. senha // Char com 20 posições e não pode ser nulo. Obs: Senha para acessar o endereço_do_site
 5. status // Campo lógico e não pode ser nulo Obs: Usado para saber o se o site está conectado
9. expediente_do_medico_data // Chave múltipla id_medico+Data
1. id_medico // tipo inteiro e não pode ser nulo
 2. dateTime // tipo data e não pode ser nulo
10. expediente_do_medico_horas // Chave múltipla id_medico+Data+Hora_inicial
1. id_Medico // tipo inteiro e não pode ser nulo
 2. dateTime_inicial // Data e Hora inicial do expediente do médico e não pode ser nulo
 3. dateTime_final // Data e Hora final do expediente do médico e não pode ser nulo
11. agenda
1. id // tipo inteiro serial
 2. id_Medico // tipo inteiro e não pode ser nulo
 3. id_Cliente // tipo inteiro e não pode ser nulo
 4. dateTime // Data e hora prevista da consulta e não pode ser nulo
 5. dateTime_confirmação // Data e hora da confirmação da consulta e não pode ser nulo
 6. id_convênio // tipo inteiro e pode ser nulo caso id_formas_de_pagamento não for nulo.
 7. id_status_da_agenda_ou_consulta // tipo inteiro e não pode ser nulo
 8. id_formas_de_pagamento // tipo inteiro e pode ser nulo caso id_convênio não for nulo.
 9. dateTime // Data e hora em que a agenda foi criada e não pode ser nulo

10. observações // Char com 255 posições e pode ser nulo

12. formas_de_pagamento

1. id // tipo inteiro serialNatureza
2. Nome // Char com 30 posições e não pode ser nulo. Obs: Pode ser:
0 - Dinheiro; 1 - Pix; 2 - Convênio; 3 - Cartão de debito; 4 - Catão de credito Mastercard; 5 - Catão de credito Visa; etc....

13. consulta //Usado para baixar a agenda

1. id // tipo inteiro serial
2. id_agenda // tipo inteiro e pode ser nulo. Obs: Uma consulta não é obrigado ter agenda.
3. if_cliente // tipo inteiro e não pode ser nulo.
4. id_medico // tipo inteiro e não pode ser nulo.
5. dateTime // Data e hora da consulta e não pode ser nulo
6. id_convênio // tipo inteiro e pode ser nulo. Obs: Uma consulta não é obrigado ter convênio
7. id_forma_de_pagamento // tipo inteiro e pode ser nulo se id_convênio não for nulo.
8. id_status_da_agenda_ou_consulta // tipo inteiro e não pode ser nulo
9. Observação // Char com 255 posições e pode ser nulo

3. Criar rotinas para:

1. Criar programa para coletar as informações do paciente

1. Obs?

1. Quais textos utilizar para interagir com o cliente no WhatsApp?

1. Podemos usar a IA para textos humanizados.
2. Outras forma é fazer perguntas e o cliente responde com o número da opção.

2. Cadastros necessários para comunicar-se com os clientes:

1. Medico
2. Convênio
3. Serviço_de_agendas
4. Integração
5. Expediente_do_medico_data
6. Expediente_do_medico_horas

3. Criar programa para confirmações de consultas.

1. Cadastros atualizados aqui:

1. clientes
2. Agenda
3. Disponibilidade_do_Paciente

4. Confirmação de Consultas:

1. Utilize a *API do WhatsApp* para enviar

2. Implementar a *API de voz da OpenAI* para fazer *chamadas de voz automatizadas para confirmação de consultas*.

5. Banco de Dados de Informações Adicionais:

1. Criar um *banco de dados adicional* para armazenar informações sobre *convênios, preços de consultas e hospitais onde o médico atende*.

6. Testes e Validação:

1. Realizar *testes abrangentes* para garantir o funcionamento adequado de todas as funcionalidades.

7. Documentação e Treinamento:

1. Preparar *documentação detalhada* e *oferecer treinamento* para o *médico* e a *equipe* sobre o uso da *secretária virtual*.

8. Manutenção e Atualização Contínua:

1. Criar um *plano de manutenção e atualização* para manter a segurança, eficiência e compatibilidade do sistema.

